



Improving Weather Prediction in the Himalayas: Automated Machine Learning Approach for Joshimath, Uttarakhand

Anurag Choubey¹ · Ayush Mittal² · Shivendu Mishra³ ·
Prince Rajpoot³ · Bollampalli Areen Reddy⁴ ·
Amit Kumar Pandey⁵ · Ritika Yaduvanshi³

Received: 3 February 2025 / Accepted: 27 June 2026 / Published online: 8 July 2026
© The Institution of Engineers (India) 2026

Abstract This work highlights the significance of minimizing human intervention for precise precipitation forecasting in Joshimath, Uttarakhand, India, a region prone to heavy rainfall, landslides, and seismic activity. The study uses a correlation matrix to identify relationships between climatic variables and precipitation intensity and develops forecasting models with Automated Machine Learning (AutoML), time series transformers (Informer, Autoformer), and nine traditional ML models (Random Forest (RF), XGBoost, Decision Tree, Bagging, AdaBoost, Ridge Regression, K-Nearest Neighbors, Multiple Linear Regression, and Lasso Regression). Leveraging 14 years of data from Visual Crossing, all models' performance is evaluated using MAE, RMSE, and R^2 . The findings indicate that the AutoML (H2O) model does better than others (RMSE: 5.8600, MAE: 1.9787, R^2 : 58.01 %), eliminating the need for manual model selection and hyperparameter tuning, making

it a more efficient alternative to traditional machine learning methods.

Keywords AutoML · Environment · Joshimath · Grid search · Machine learning · Weather prediction

Introduction

About 51% of all landslides in India between 1800 and 2011 happened in the North-West Himalayas, which has the most hazardous terrain and is the location of recurring landslides [1–3]. Joshimath, the subject of this research, has been steadily sinking for decades and is vulnerable to earthquakes and floods [4, 5]. Recent research highlights the impact of climate change on Joshimath, attributing it to the significant increase in anthropogenic activities [6]. Accurate rainfall

✉ Shivendu Mishra
shivendu0584@gmail.com
Anurag Choubey
anurag.pcs17@iitp.ac.in
Ayush Mittal
amittal@recabn.ac.in
Prince Rajpoot
princeraj@recabn.ac.in
Bollampalli Areen Reddy
bollampalli2101cs19@iitp.ac.in
Amit Kumar Pandey
amitkumarpandey1@gmail.com
Ritika Yaduvanshi
dinkal.ritika@gmail.com

¹ School of Computer Science Engineering and Technology,
Bennett University, Greater Noida, Uttar Pradesh 201310,
India

² Department of Civil Engineering, Rajkiya Engineering
College Ambedkar Nagar, Ambedkar Nagar,
Uttar Pradesh 224122, India

³ Department of Information Technology, Rajkiya
Engineering College Ambedkar Nagar, Ambedkar Nagar,
Uttar Pradesh 224122, India

⁴ Department of Computer Science and Engineering, Indian
Institute of Technology, Patna, Bihar 801106, India

⁵ Department of Applied Science and Humanities, Rajkiya
Engineering College Ambedkar Nagar, Ambedkar Nagar,
Uttar Pradesh 224122, India

forecasting and identification of precipitation patterns are crucial for implementing effective mitigation strategies. This area has become a vital focus in data science and machine learning, employing advanced computational techniques and data analysis to enhance understanding and improve forecast accuracy, benefiting from optimizing water resource utilization, maximizing crop yields, and mitigating risks associated with floods and rain-related illnesses, leading to reduced mortality rates. Further advancements in this field are expected to enable more effective planning and decision-making amid changing weather patterns.

Previous researchers have employed meteorological data, data mining techniques, and big data analysis to estimate daily, monthly, and annual rainfall quantities [7]. To forecast rainfall in Assam, authors Dutta and Tahbiller [8] employed a data-mining tactic, specifically utilizing a multiple linear regression (MLR) model. Their model incorporated maximum and minimum temperatures, wind speed, and mean sea level variables. But they did not include wind direction in their statistical model, which could improve prediction accuracy. In contrast, using tipping-bucket data, Authors Kusiak and Verma [9] explored rainfall forecasting in a watershed basin near Oxford, Iowa. Their study compared five data mining algorithms and found that the support vector machine (SVM) outpaced the others, especially for longer lead periods. Authors Trenberth [10] emphasized the significance of atmospheric factors such as wind and sunlight in influencing rainfall patterns, contributing to improved understanding of precipitation behavior. Authors Tharun and Prakash [11] motivated on forecasting rain severity in Coonoor, Tamil Nadu, utilizing regression analysis and statistical modeling. By leveraging daily weather observations, their research trained regression models, including Random Forest, Decision Tree, and Support Vector Regression. The Random Forest model displayed the highest performance among the tested models.

Furthermore, using variables such as changes in minimum and maximum temperatures, rainfall, wind speed, and evaporation, authors Chowder and Girisha [12] employed data mining methods such as clustering and classification. Their work aimed to better understand variations in weather and climatic characteristics, such as rainfall and humidity. They advised using predictive analytics algorithms and high-resolution data collection for improved precision. With the aim of enhancing the accuracy of water demand predictions, author Zhao [13] established a model to anticipate the water demand of ecological environments, employing tools for data visualization, data mining, predictive analysis, semantic engines, storage, management, and warehousing. Authors Zhou and Ruan [14] conducted a spatial risk assessment in China's marine transportation sector using machine learning (ML) and geographical big data. Their results emphasized that the XGBoost algorithm outstripped

others in forecasting potential hazards. Authors Reddy and Babu [15] conducted a survey of neural networks and found that while they are effective for annual forecasts, their accuracy decreases when applied to modest datasets or used frequently. Authors Kundu and Biswas [16] examined the effectiveness of ensemble learning systems for rainfall forecasting and concluded that these systems outperform traditional models. Authors Latif and Hazrin [17] declared the efficacy of machine learning algorithms in rainfall prediction, particularly when supplemented with remote sensing data, which improves forecast accuracy. Moreover, authors in [18, 19] recognized four meteorological variables, relative humidity, pressure, wind speed, and temperature, crucial for reliable rainfall predictions.

Over the past few years, there has been a surge in the growth of various ML models to increase rainfall forecast accuracy. Authors in [20, 21] have made distinguished contributions in this domain. In a recent review, Mohammad and Batelaan emphasized the effectiveness of ensemble learning methods in hydrology [22]. Choubin and Moradi [23] have unveiled the remarkable potential of MDA (multivariate discriminant analysis) in accurately predicting flood vulnerability using 11 key parameters. The study also utilized classification, regression trees, and support vector machines (SVM) as part of its methodology. The amount of rainfall is influenced by various factors, including cloud cover, wind speed and direction, humidity, temperature, water level, solar radiation, solar energy, ultraviolet index, and other related variables [24]. A study by Nti and Zaman [25] demonstrates that XGBoost and other gradient tree-based ensemble models perform better than AdaBoost and Random Forest in predicting crop suitability and productivity. The models outlined here only include a small number of variables: temperature, humidity, rainfall, pH, nitrogen, and phosphorus. Solar radiation, visible water vapour, and other critical elements for rainfall forecasting were identified by [26] using a linear regression model.

Recently, applying transformers to time series forecasting in environmental settings has proven the increasing capability of transformer architecture in dealing with spatiotemporal relationships. Recent research has developed transformer approaches in the context of environmental time series forecasting and hydrology, where their advantages over conventional methods have been clearly demonstrated [27, 28]. In addition, transformer-enabled models have been effectively utilized for precipitation forecasting and downscaling with increased accuracy [29, 30]. In addition, recent works on precipitation nowcasting using transformer-based generative and spatiotemporal models have been included, highlighting the role of AI-driven approaches in disaster preparedness and environmental monitoring [31]. These publications point out the fast development of Environmental AI, where deep

learning and transformers are widely applied to perform climate modeling and weather forecasting. Also, recent advances in automatic machine learning have been taken into account in order to emphasize its importance for the selection of proper algorithms and efficient hyperparameter tuning [32, 33]. However, many existing studies have not effectively utilized advanced ensemble learning algorithms (such as bagging, boosting techniques like Ada-Boost and XGBoost, random forests, and decision trees) or grid search methods to improve rainfall forecasting precision. Furthermore, the influence of various environmental factors on rainfall prediction is often overlooked. Additionally, Manual model selection and hyperparameter tuning in traditional approaches are time-consuming and demand domain expertise, leading to inconsistent results across different datasets. To address these challenges, this work leverages AutoML models and compares them with traditional ML models and time series transformer models, making the following key contributions

- i. Analyzing 18 key parameters to leverage advanced machine learning techniques, including AutoML, time series transformer models (Autoformer, Informer, and PatchTST), and traditional ML models (Random Forest, XGBoost, Decision Tree, Bagging, Ada Boost, Ridge Regression, Multiple Linear Regression, Lasso Regression, and K-Nearest Neighbor), for accurate rainfall prediction
- ii. Identify the principal meteorological variables influencing rainfall patterns and develop accurate precipitation predictions.
- iii. Data from Joshimath spanning the past 14 years (2010–2023) is preprocessed using Visual Crossing. A correlation matrix is then used to analyze the relationships between preprocessed features and rainfall variables.
- iv. Assesses traditional ML model accuracy through grid search optimization and robustness testing, ensuring reliable performance metrics for precipitation prediction.
- v. Demonstrates that the AutoML (H2O) model significantly outperforms traditional ML models in RMSE, MAE, and R^2 , establishing it as a superior approach for minimizing human intervention in model selection and hyper-parameter tuning

The remaining article is structured below: Section 2 describes the used machine learning algorithms and statistical evaluation. Section 3 outlines the data collection and methodology used. Section 4 provides an in-depth analysis of weather statistics and correlations. Section 5 presents the experimental results obtained from the work. Section 6 presents the discussion, and finally, Section 6 concludes the paper.

ML Algorithms and Statistical Evaluation Indicators

ML is a branch of artificial intelligence that learns by itself without requiring explicit programming. Machine learning models are vital for deriving information from data and making automated decisions in multiple industries [34, 35]. In this section, some of the most widely known ML algorithms employed for building the model and evaluating its prediction results are briefly described.

ML Algorithms

The various advanced and traditional ML models used in this paper are described below:

AutoML

The main objective of autoML is automatic selection of the most suitable ML framework depending on the data by algorithm selection or meta-learning. This is done through the trial of different algorithms such as SVMs, neural networks, random forests, and decision trees, among others. AutoML frameworks incorporate libraries of pre-implemented algorithms, enabling systematic examination of various models and configurations [36]. Recently, AutoML models such as H2O [37], Auto-Sklearn [38], AutoTS [39], AutoGluon [40], TPOT [41, 42], AutoKeras [43], and Auto-PyTorch [44] have emerged, mixing feature engineering, model selection, and hyperparameter optimization. The proposed work uses H2O, AutoKeras, and TPOT models to accurately predict rainfall. The subsequent sections review and compare these models.

- **H2O[37]:** It is a distributed, open-source, and scalable supervised learning framework within the H2O library. It is widely used in academia and industry. It facilitates regression, multi-class classification, and binary classification. Its key advantage lies in rapidly generating predictions using multiple H2O models. With APIs in various programming languages, the H2O model is accessible and versatile across different domains. The framework uses ML algorithms such as gradient-boosting machines, generalized linear models, random forests, and deep learning. It uses Bayesian optimization, grid, and random search to automate model selection and hyperparameter tuning. Thus, end-to-end automation and interoperability across different setup formats streamline model building and deployment, making the H2O model useful for both beginners and professional data scientists.
- **Autokeras[43]:** It is an AutoML framework built on the Keras API that prioritizes deep learning models over traditional algorithms. It utilizes a neural architecture search

methodically to investigate and determine the best neural network architectures for a specific modelling mission, distinguishing it from other AutoML frameworks.

- **TPOT**[41, 42]: This machine learning platform uses a genetic algorithm to create and optimize pipelines automatically. The platform supports model selection across decision trees, KNN, XGBoost, random forests, and logistic regression, and incorporates feature selection techniques such as standard scaling, SelectKBest, randomized PCA, and recursive feature elimination. The procedure entails creating and testing 100 tree-based pipelines, selecting the top 20 from each generation, and replicating and changing them to produce a fresh batch of 100 pipelines. This cycle is repeated 100 times, resulting in the top-performing models from the final generation.

Time Series Transformers

Transformer-based networks, including Autoformer, Informer, and PatchTST is described in this section.

- **Autoformer**[45]: It is considered for time-series forecasting, particularly effective for long-term predictions and seasonal patterns. It exploits an advanced auto-correlation mechanism to capture short-term and long-term dependencies, improving computational efficiency and model performance. The encoder divides data from time series into pattern and seasonal components, empowering the model to focus on recurring patterns while filtering out noise. The decoder progressively integrates trend information, improving the model's ability to detect both short-term seasonal variations and long-term trends.
- **Informer**[46]: It addresses limitations of standard Transformers by introducing ProbSparse Self-attention for reduced memory usage and computational time, self-attention distilling to downsample dimensions, and a generative decoder that outputs long sequences in a single step, minimizing cumulative errors.
- **PatchTST**[47]: It incorporates two key features: (1) Patching, which segments time series data into smaller sub-sequences (patches) as input tokens, and (2) Channel Independence, treating each variable as a separate univariate series with shared transformer weights and embedding processes across channels.

eXtreme Gradient Boosting (XGBoost)

XGBoost is a variant of the Gradient Boosting approach that utilizes more precise estimates to achieve the optimal tree model [48]. Regularization techniques play a crucial role in mitigating the issue of overfitting and enhancing the overall generalization performance, as highlighted by Nti and Zaman [25]. XGBoost has become prevalent among

writers due to its notable performance in terms of algorithmic speed and prediction accuracy. This has led to extensive adoption in addressing various regression and classification problems [49, 50]. The gradient descent method is highly effective, encompassing both a linear model algorithm and a tree learning algorithm, as discussed by Srinivas and Somula [51].

The XGBoost framework incorporates techniques for computing features' importance, as shown in Fig. 1. It is important to acknowledge that various factors can influence these features, including the type of data and its cardinality [52].

Multiple Linear Regression (MLR)

Regression analysis is a statistical approach for studying and modelling variables' relationships. A multiple regression model includes more than one regressor variable. The equation shown below represents the model with k regressors.

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_k x_{ik} + \epsilon_i \quad (1)$$

The regression coefficients are represented by the parameters β_k , where $k = 0, 1, \dots, k$, and the error term is denoted as ϵ_i [53]. Figure 2(a) represents the 3-D plot of the regression model of the given equation, and Fig. 2(b) shows the contour plot of variables x_1 and x_2 .

Random Forest (RF)

Random Forests Algorithm is one such machine learning algorithm that uses a combination of multiple decision trees to leverage the benefits provided by all of them. First introduced in [54], Random Forests can be used for both classification and regression purposes. In the Random Forests model, each decision tree is constructed by randomly choosing some features from the whole feature set and training data [55]. Random Forest combines the forecasts of all constituent trees, usually by majority voting for classification and averaging for regression [54, 56], thereby achieving greater predictive power and robustness than individual decision trees. Here, different sets of predictors are used for each training dataset to reduce variance and achieve optimal results. When compared to a single decision tree, the accuracy and robustness of the random forest prediction are significantly improved. This is accomplished by combining the predictions of all individual trees, which is frequently accomplished using majority voting for classification and averaging for regression [55, 56], as illustrated in Fig. 3, which shows that a different set of predictors is used in each training dataset to reduce variance and achieve good results.

Fig. 1 Feature importance for XGBoost algorithm on weather dataset of Joshimath (Uttarakhand)

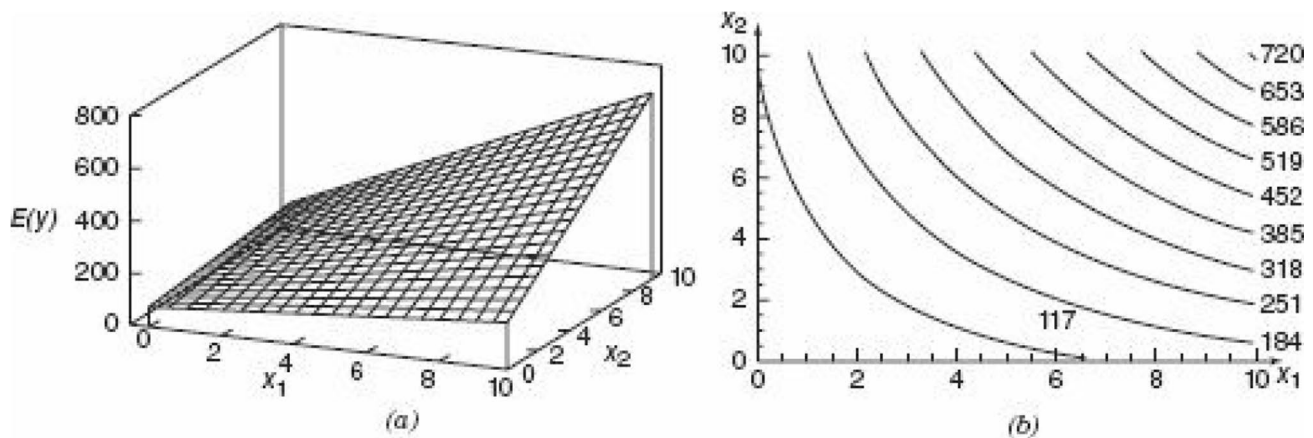
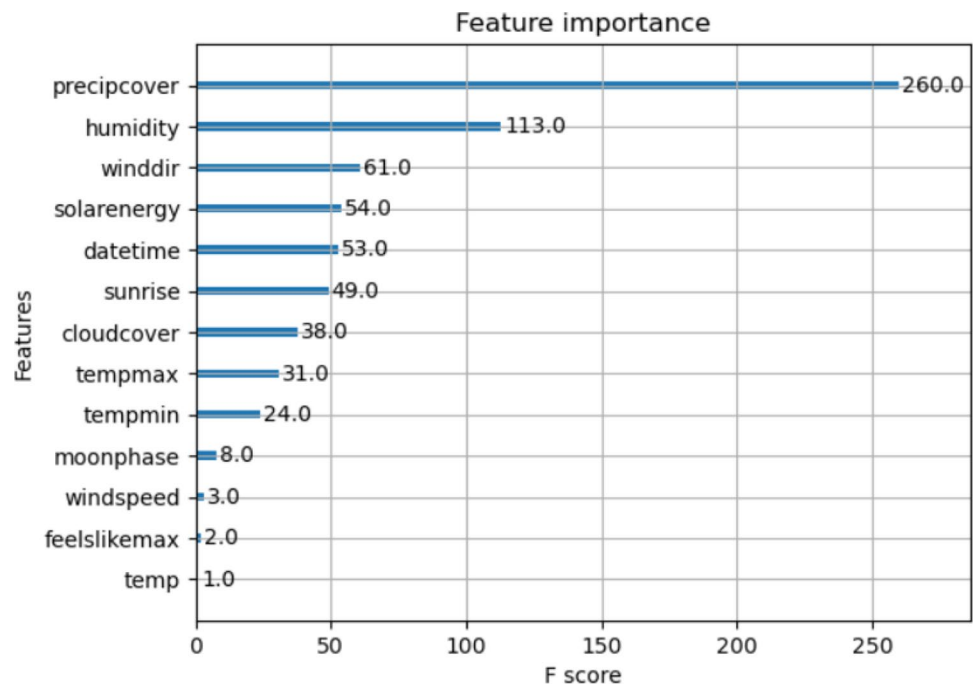


Fig. 2 **a** Plot of the regression model $E(y) = 50 + 10x + 7x_2 + 5x_1x_2$. **b** Contour plot[53]

Decision Tree

For modelling continuous target variables using a collection of attributes, decision tree regression offers a straightforward and easy-to-understand non-parametric methodology [57]. The data is first partitioned at each node according to the best split of a given feature, which gives less error than other splits; the structure resembles a tree. Eventually, it reaches leaf nodes that contain projected values for the target variable. By reducing this basic framework to a series of “if-then” rules, valuable information on the relationships between attributes and the desired result may be uncovered [58]. Applications like consumer behaviour analysis, where knowing the data’s structure is

crucial, are perfect for decision tree regression because of its interpretability [59]. Figure 4 represents the decision tree obtained from Joshimath’s weather data set.

AdaBoost

The concept of AdaBoost is based on ensemble learning, which combines many models to enhance the correctness and effectiveness of a single model. Additionally, it can deal with outliers, missing variables, and regression and classification issues. It combines several weak classifiers into one powerful model, as shown in Fig. 5 [22].

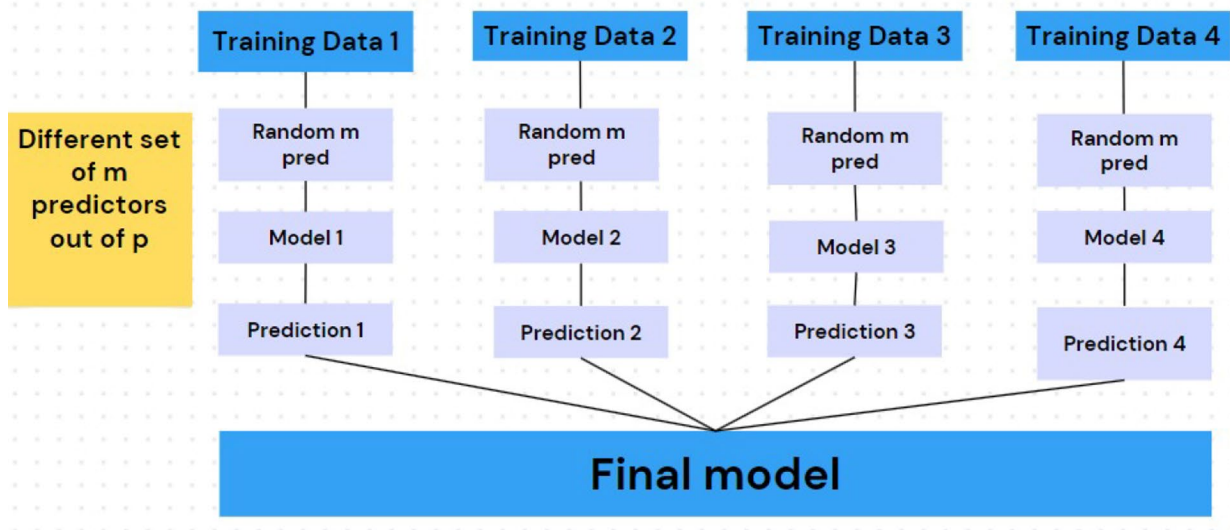


Fig. 3 Working of random forest algorithm

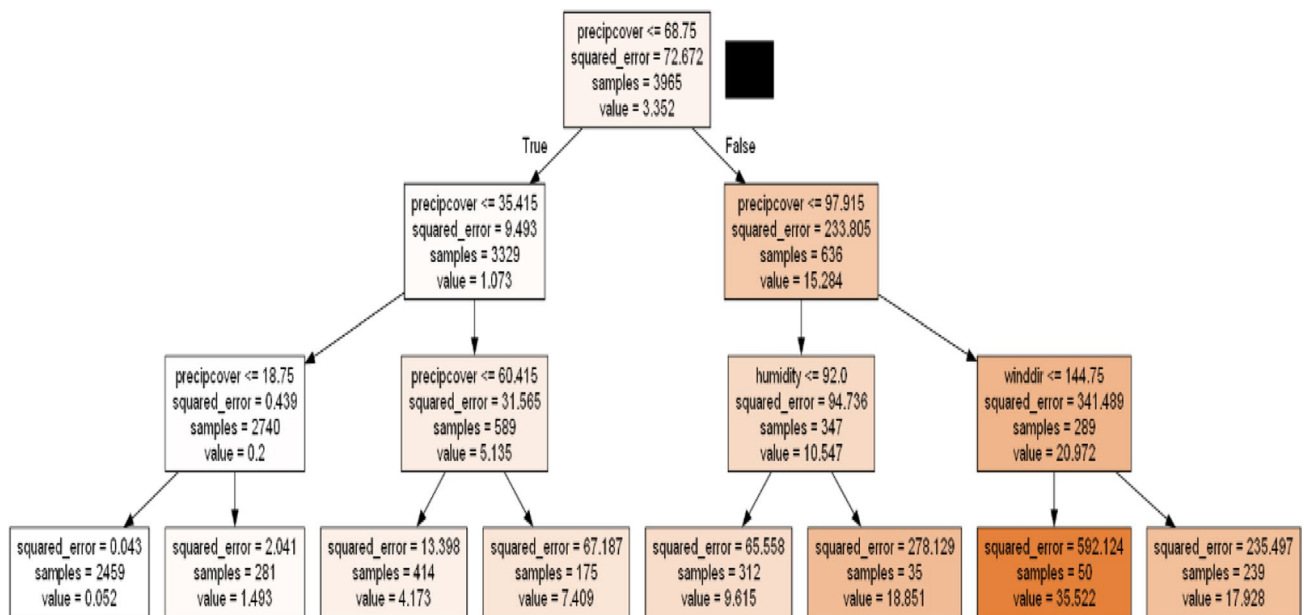
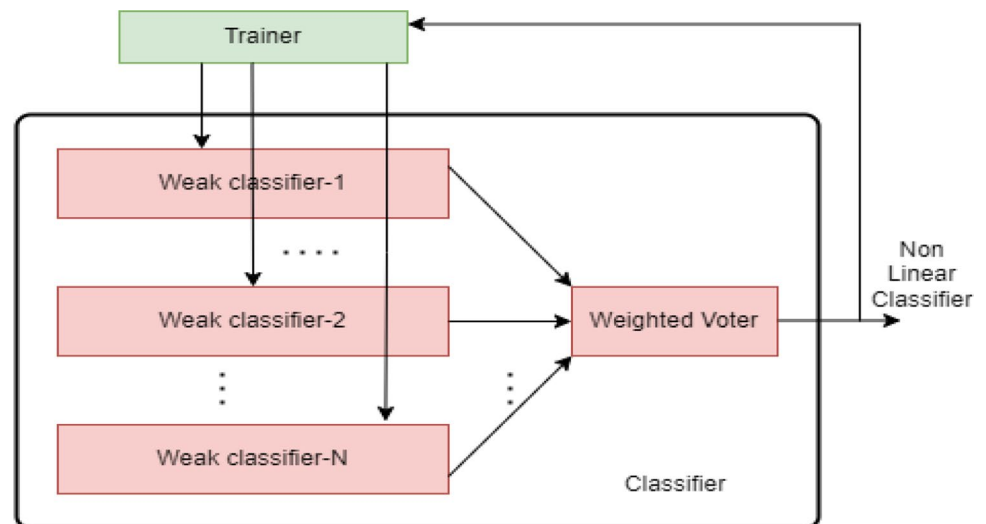


Fig. 4 Decision tree for weather dataset of Joshimath, Uttarakhand

K-Nearest Neighbor (KNN)

Using Minkowski, Manhattan, and Euclidean distances for data classification, KNN is a non-parametric learning method [60]. Regression and classification issues may be addressed using the KNN approach. Class membership results from classification, whereas the average of the k

closest neighbours' values results from regression. Using fewer features improves KNN's performance [61, 62]. This approach is called a lazy learner algorithm because instead of producing predictions immediately, it saves the dataset and sorts it according to similarities before doing anything with it [63].

Fig. 5 Ada boost model

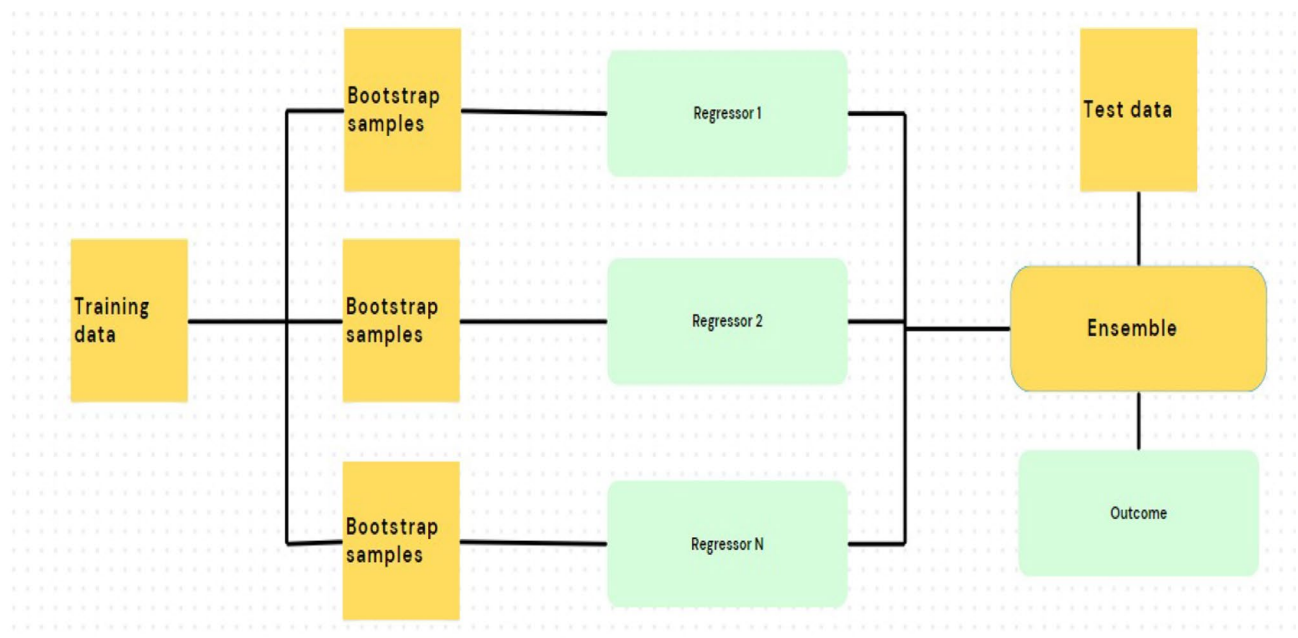
Bagging

Combining several weak learners into one more accurate prediction is the goal of bagging, an ensemble method in ML (sometimes termed Bootstrap Aggregating). Bagging involves training multiple decision trees with different bootstrap samples from the training data [64]. Data points could not show up at all or many times in a single bootstrap sample. The reason is that bootstrap samples are generated by randomly selecting a fraction of the original training data and then replacing it. Figure 6 shows the process of the bagging method.

Bagging has improved model accuracy, stability, and resilience against outliers in empirical research from several domains. For instance, authors in [65] found that bagged decision trees performed better than single decision trees in several classification problems, and Breiman [66] found that bagged learners were more stable against overfitting.

Lasso

Blending L1 regularization with linear regression, lasso regression, proposed by Tibshirani [67], is a powerful machine learning tool. This approach does more than estimate the

**Fig. 6** Bagging model

relationships between the features and the target variable. It also picks the features, maybe getting rid of features of low importance by setting their coefficients to zero or decreasing them somewhat so that the error is minimal.

$$RSS = RSS + \lambda \sum_{j=1}^p |\beta_j| \quad (2)$$

Where, RSS = Residual sum of squares, β_j = coefficient of variables, and λ = tuning parameter.

The regularization parameter λ may be varied to alter the model's shrinkage and sparsity levels. More shrinkage occurs with higher λ values, leading to more features, but there is also a greater danger of biasing the coefficients of critical features [68]. It's vital to remember that Lasso may not be the best choice for projects that include crucial feature interactions or all features are equally significant [69].

Ridge

Ridge regression discourages overfitting by reducing the weight coefficients towards zero. This penalty, proportional to the sum of squared feature weights multiplied by a hyperparameter λ , prevents the model from overfitting the training set [70]. When applied to unknown data, ridge regression models achieve less volatility and better performance. Ridge regression is a powerful tool in linear regression that combats overfitting and multicollinearity [71].

$$RSS = RSS + \lambda \sum_{j=1}^p \alpha_j^2 \quad (3)$$

Where RSS = Residual sum of squares, α_j = coefficient of variables, and λ = tuning parameter.

Grid Search for Hyperparameters Tuning

A set of parameters controls the execution of many ML algorithms. Before the algorithm's learning phase, these parameters are established; they are called hyperparameters. Certain hyperparameters are adjusted to enhance the traditional model's performance and accuracy in predicting the unknown dataset. This study uses a grid search method based on cross-validation (CV) to enhance the hyperparameters of the algorithms discussed before [63]. Hyperparameters that can be altered using grid search and their common choices that give the best results of XGBoost, Random Forest, Ada Boost, Bagging, Decision Tree, MLR, KNN, Ridge, and Lasso are depicted in Tables 1, 2, 3, 4, 5, 6, 7, 8, 9, respectively. Furthermore, Table 10 provides the key hyperparameters considered for the implementation of the time-series transformer models - namely Informer, Autoformer, and PatchTST. These models are trained using an input length of 96 for the time series data, along with a prediction horizon spanning between 1 to 7 timesteps for multi-step forecasting purposes. In particular, the models are trained using the Adam optimization algorithm with a learning rate of 0.0001 to ensure steady convergence and for 200 epochs using a batch size of 32. Further, a context length of 12 is adopted for determining the temporal length provided to the encoder layer. Finally, the analyzed transformers have varying computational complexities due to differences in their architectures. The computational cost mainly depends on three values: the context length (L), the patch length (P), and the stride length (S). These values are used for estimating the computational complexities of Informer ($\mathcal{O}(L \log L)$), Autoformer ($\mathcal{O}(L)$), and PatchTST ($\mathcal{O}(N^2)$). In the latter model, N is the number of patches, which equals: $N = \left\lfloor \frac{L-P}{S} \right\rfloor + 2$. Importantly, the same tun-

Table 1 Hyperparameters of the XGBoost algorithm

Hyperparameter	Description	Common Choices
n_estimators	Total number of trees used for the ensemble	Integer (e.g., 100, 500, 1000), often determined through tuning
max_depth	Limit of each tree's depth	Integer (e.g., 3, 6, 9), lower values for simpler models, higher values for more complex problems
learning_rate	The use of step size shrinking to avoid overfitting	Float (e.g., 0.01, 0.1, 0.3), smaller values for slower convergence and potential better accuracy
colsample_bytree	Ratio of attributes to take into account per tree	Float (e.g., 0.5, 0.8, 1.0), lower values for regularization and avoiding overfitting
subsample	Number of observations to use for training each tree	Float (e.g., 0.5, 0.8, 1.0), lower values for regularization and faster training
gamma	Level of loss reduction necessary to do one more partition on a leaf node	Float (e.g., 0, 0.1, 0.5), higher values for more conservative model growth
reg_lambda	Level 2 regularization phrase used to weights	Float (e.g., 0, 1, 10), higher values for stronger regularization
reg_alpha	L1 regularization term applied to weights will	Float (e.g., 0, 1, 10), higher values for stronger regularization

Table 2 Hyperparameters of Random Forest Algorithm

Hyperparameter	Description	Common Choices
n_estimators	The forest's tree count	Integer (e.g., 100, 500, 1000), often determined through tuning
bootstrap	Whether to use bootstrap sampling	Boolean (True or False)
max_depth	Max depth of each tree	Integer (e.g., 3, 6, 9), lower values for simpler models, higher values for more complex problems
max_features	Features to take into account for each split	'auto' (sqrt(n_features)), 'log2', int (number of features)
criterion	A function to assess a split's quality	'gini' (default for classification), 'entropy.'
min_samples_split	The bare minimum of samples needed to divide an internal node	Integer (e.g., 2, 5, 10)
random_state	Seed for random number generator	Integer (for reproducibility)
min_samples_leaf	The bare minimum of samples needed at a leaf node	integer (e.g., 1, 2, 4)

Table 3 Hyperparameters of the AdaBoost algorithm

Hyperparameter	Description	Common Choices
n_estimators	The number of base classifiers, or weak learners, to use	Integer (e.g., 50, 100, 500)
learning_rate	Scaling factor for updating weights after each iteration	Float (e.g., 0.1, 0.5, 1.0), smaller values can lead to slower convergence but potentially better generalization
algorithm	Base classifier algorithm to use	'SAMME' (discrete boosting), 'SAMME.R' (real-valued boosting)
random_state	The random number generator seed	Integer (for reproducibility)

Table 4 Hyperparameters of Bagging Algorithm

Hyperparameter	Description	Common Choices
n_estimators	Number of base estimators (trees) in the ensemble	Integer (e.g., 100, 500, 1000)
max_samples	Ratio or absolute number of samples to draw for training each base estimator	Float between 0 and 1 (e.g., 0.5, 0.8), or int (number of samples)
max_features	Ratio or absolute number of features to consider for each split in each base estimator	Float between 0 and 1 (e.g., 0.5, 0.8), or int (number of features)
bootstrap	Whether to use bootstrap sampling (random sampling with replacement)	Boolean (True or False)
bootstrap_features	Whether to use bootstrap sampling for features	Boolean (True or False)
random_state	The random number generator seed	Integer

Table 5 Hyperparameters of the Decision Tree Algorithm

Hyperparameter	Description	Common Choices
criterion	A function to assess a split's quality	'gini' (default for classification), 'entropy.'
random_state	The random number generator seed	Integer (for reproducibility)
splitter	strategy for choosing the split at each node	'best' (choose the best split), 'random' (choose a random split)
min_samples_split	The bare minimum of samples needed to divide an internal node	Integer (e.g., 2, 5, 10)
max_depth	Max depth of each tree	Integer (e.g., 3, 6, 9), lower values for simpler models, higher values for more complex problems
max_features	Features to take into account for each split	'auto' (default, sqrt(n_features)), 'log2', int (number of features)
min_samples_leaf	The bare minimum of samples needed at a leaf node	integer (e.g., 1, 2, 4)

Table 6 Hyperparameters of the MLR algorithm

Hyperparameter	Description
fit_intercept	Whether to include a constant term (intercept) in the model. The default is true
n_jobs	Quantity of cores used for parallel processing. 1 is the default
normalize	Whether to normalize the input features by subtracting their mean and dividing by their standard deviation. The default is false
copy_X	Whether to copy the input X matrix. The default is true

Table 7 Hyperparameters of the K-NN algorithm

Hyperparameter	Description	Common Choices
n_neighbors	Number of nearest neighbors to consider for voting	Odd numbers (e.g., 3, 5, 7, 9), often starting with 5 and adjusting based on performance
algorithm	Algorithm for finding nearest neighbors	'auto' (chooses best algorithm based on dataset), 'ball_tree', 'kd_tree', 'brute.'
metric	Distance metric for calculating similarity between data points	Euclidean (default), Manhattan, Minkowski, Cosine
weights	Weighting scheme for neighbors' votes	'uniform' (all neighbors have equal weight), 'distance' (closer neighbors have more weight)
leaf_size	Leaf size for tree-based algorithms (ball_tree, kd_tree)	Affects memory usage and speed, often set to 30
p	Parameter for Minkowski distance metric (when metric='minkowski')	Usually set to 2 (Euclidean) or 1 (Manhattan)

Table 8 Hyperparameters of the Ridge Algorithm

Hyperparameter	Description	Common Choices
alpha	Regularization strength, control the penalty for large coefficients	Float (≥ 0), often determined through tuning, higher values for stronger regularization
fit_intercept	Whether to incorporate bias, or an intercept term, into the model	Boolean (True or False)
normalize	Whether to normalize features by their L2 norm before fitting	Boolean (True or False), can improve convergence speed
copy_X	Whether to copy the data X internally	Boolean (True or False), set to True if you want to modify X after calling fit
tol	Tolerance for stopping criteria	Float, convergence tolerance for iterative solver
solver	Algorithm to use for optimization	'auto' (default, chooses best based on data), 'svd', 'cholesky', 'lsqr', 'sparse_cg', 'sag', 'saga'
random_state	The random number generator seed	Integer (for reproducibility)

Table 9 Hyperparameters of the Lasso algorithm

Hyperparameter	Description	Common Choices
alpha	Regularization strength; higher values lead to more coefficients being set to zero	Float (e.g., 0.1, 1.0, 10.0)
max_iter	The solver's maximum number of iterations	Integer (e.g., 100, 1000, 5000)
tol	Tolerance for convergence	Float (e.g., 1e-4, 1e-6)
random_state	The random number generator seed (for reproducibility)	Integer
selection	Algorithm for performing feature selection	'cyclic' (default), 'random'

ing strategy and evaluation protocol were consistently applied across all models to ensure a fair comparison, model-specific optimization, and transparency in the

experimental setup. Hyperparameter configurations that correspond to the best performance while maintaining model stability are chosen as the final.

Table 10 Hyperparameters of the time series transformer models, i.e., Informer, Autoformer, and PatchTST

Hyperparameter	Description	Values Used
Input Sequence Length	Total length of sequence prepared in data tokenization step	96
Prediction Horizon	Prediction length for model decoder	[1,2,3,4,5,6,7]
Optimizer used	An algorithm to minimize loss during training	Adam
Epochs	Defines the no. of times the model processes the entire training set	200
Learning Rate	Defines the step size during weight update	0.0001
Batch Size	No. of time series sequences processed simultaneously	32
Context Length (L)	The exact time step length passed to the encoder	12
computational cost	Depends on models' tokenization strategy	Informer - $O(L \log L)$, Autoformer - $O(L)$, PatchTST - $O(N^2)$

Table 11 Performance metrics

Parameter	Mathematical Formula	Ideal Value
R^2 (R-squared)	$1 - \frac{RSS}{TSS}$	1.0 (perfect fit)
RMSE (Root Mean Squared Error)	$\sqrt{\frac{\sum (a_i - \hat{a}_i)^2}{n}}$	0.0 (no error)
MAE (Mean Absolute Error)	$\frac{\sum a_i - \hat{a}_i }{n}$	0.0

Statistical Performance Measurement Indices

To evaluate the accuracy of each model, several statistical performance parameters are available. R^2 -Score, RMSE, and MAE are performance measures used in this study. Lower RMSE and MAE mean fewer prediction errors, whereas R^2 -Score (explained variance) should ideally approach 1 (perfect fit). MAE is easily interpretable and less affected by outliers, unlike RMSE, which penalizes large errors. Table 11 displays the efficiency matrix for evaluating ML models.

where n is the number of assessments in the dataset. The RSS stands for Residual Sum of Squares, while the TSS stands for Total Sum of Squares. The a_i represents the actual value of the dependent variable in the i th observation, whereas the $\hat{a}_i$ represents the predicted value of the dependent variable in i th observation.

Data Collection and Methodology Adopted

Study Location

Joshimath is situated on a sand and stone deposit, which is very vulnerable. An old landslide formed its foundation. Undercutting by the Alaknanda and Dhauliganga river currents further contributes to the occurrence of natural calamities [72]. The geographic location of the research area is depicted in Fig. 8.

Data Collection

The meteorological data for the last 14 years (i.e., 2010–2023) obtained from Visual Crossing of Joshimath, Uttarakhand, has been used [73]. The screenshot of a few rows of the data set is shown in Fig. 7

Methodology Adopted

To effectively construct and assess models, the dataset was divided into two separate subsets: a testing set that included 20% of the data and a training set that included 80%. The training set serves as the basis for the model's learning, providing ample examples for refinement. In contrast, the testing set remains impartial, serving as a critical evaluator of the model's actual effectiveness. Subsequently, the model is evaluated on the test set, identifying opportunities for improvement and assessing its accuracy and precision.

Weather Statistics and Correlation Analysis

The statistics for all rainfall-related factors derived from the dataset are displayed in Table 12. The dataset covers a broad spectrum of precipitation, ranging from 0 mm to 151 mm, maximum temperatures spanning from -1.2°C to 34.7°C , minimum temperatures ranging from -9.2°C to 22.1°C , dew points varying from -30°C to 20.2°C , humidity levels ranging from 7.70% to 97.50%, wind directions spanning from 0.20 degrees to 359.8 degrees, cloud cover from 0% to 100%, solar energy ranging from 1 MJ/m² to 32.5 MJ/m², solar radiation varying from 11.90 W/m² to 376.20 W/m², UV index from 0 to 10, and moon phase from 0 to 0.98. These statistics identify skewness or outliers within the dataset by comparing the differences between the minimum and 25th percentile values and between the 75th percentile and maximum values of each variable. This information is useful for identifying outliers that may need to be removed to

Name	datetime	tempmax	tempmin	temp	feelslike	feelslike	feelslike	dew	humidity	precip	precippro	precipcov	precipdps	snowdept	windgust	windspeed	winddir	sealevel	cloudcov	visibility	solarradia	solarenergy	up	pincode	sewerline	name	sumet	metaphase	conditions	description	icon	stations
joshimath	01-01-2021	17.1	5.5	9.6	17.1	4.7	9.2	-14.5	18.3	0	0	0	0			14.4	126.7	28.1	175.5	15.1				7	2021-01-01T00:00:00	2021-01-01T23:45	0.01	Partly cloudy	Partly cloudy throughout the day.	partly-cloudy-day	remote	
joshimath	02-01-2021	16.6	4.3	9.5	16.6	2.7	9	-10	25.8	0	0	0	0			11.9	52.7	74.3	123.3	10.7				5	2021-01-02T00:00:00	2021-01-02T23:45	0.01	Partly cloudy	Partly cloudy throughout the day.	partly-cloudy-day	remote	
joshimath	03-01-2021	13.8	5.1	8.4	13.8	2.9	7.3	1.2	61.7	13	100	91.67	rain			11.5	64.8	47.6	108.4	9.3				4	2021-01-03T00:00:00	2021-01-03T23:45	0.05	Rain, Partly cloudy	Partly cloudy throughout the day with a shower in the afternoon.	rain	remote	
joshimath	04-01-2021	13.1	4.3	8.3	13.1	1.7	7.1	2.1	66	0.6	100	25	rain			12.6	82.4	64.7	99.2	8.5				4	2021-01-04T00:00:00	2021-01-04T23:45	0.05	Rain, Partly cloudy	Partly cloudy throughout the day with a shower in the afternoon.	rain	remote	
joshimath	05-01-2021	11.6	4.8	7.2	11.6	2.4	5.8	3.1	75.6	6.8	100	87.5	rain			12.6	77.5	61.1	81.5	6.9				7	2021-01-05T00:00:00	2021-01-05T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	06-01-2021	11.8	4	7	11.8	1.7	5.8	2.8	74.9	9.2	100	75	rain			9	54.9	32.8	106.7	9.2				7	2021-01-06T00:00:00	2021-01-06T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	07-01-2021	16.6	3.2	8.1	16.6	0.6	7.1	-6.5	37.6	0	0	0	0			9.4	73.1	0	185.2	16				7	2021-01-07T00:00:00	2021-01-07T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	08-01-2021	15.8	3.1	8.1	15.8	0.9	7.2	-8.8	30.7	0	0	0	0			9.4	55.8	0	183.7	15.8				7	2021-01-08T00:00:00	2021-01-08T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	09-01-2021	17.7	3.4	8.8	17.7	0.7	7.5	-4.1	41.5	0	0	0	0			10.4	8.1	5.6	177.2	15.2				7	2021-01-09T00:00:00	2021-01-09T23:45	0.01	Partly cloudy	Partly cloudy throughout the day.	partly-cloudy-day	remote	
joshimath	10-01-2021	18.5	3.4	9	18.5	0.8	7.8	-9.3	29	0	0	0	0			10.4	11.6	0	190.5	16.4				7	2021-01-10T00:00:00	2021-01-10T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	11-01-2021	18.4	2.6	8.8	18.4	-0.1	7.6	-11.3	25.3	0	0	0	0			13.7	16.3	0	193.5	16.5				7	2021-01-11T00:00:00	2021-01-11T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	12-01-2021	18.2	3.9	8.6	18.2	2	7.9	-10.1	28.7	0	0	0	0			16.2	303.9	0.2	191.7	16.4				7	2021-01-12T00:00:00	2021-01-12T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	13-01-2021	16.9	3.8	8.3	16.9	2.4	7.8	-7.9	33.9	0	0	0	0			14.4	281.8	28.3	190.1	16.3				7	2021-01-13T00:00:00	2021-01-13T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	14-01-2021	17.2	3.3	8.5	17.2	3.3	8.4	-7.5	34.2	0	0	0	0			15.5	255.5	27.9	182.8	15.9				8	2021-01-14T00:00:00	2021-01-14T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	15-01-2021	17.3	5	9.1	17.3	4.6	8.9	-5.1	38.5	0	0	0	0			13	275.4	37.8	155.1	13.5				7	2021-01-15T00:00:00	2021-01-15T23:45	0.01	Partly cloudy	Partly cloudy throughout the day.	partly-cloudy-day	remote	
joshimath	16-01-2021	18.4	3.9	9.4	18.4	2.4	8.7	-7.9	31.6	0	0	0	0			17.3	300.1	0	190.3	16.4				7	2021-01-16T00:00:00	2021-01-16T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	17-01-2021	17.2	6	9.7	17.2	6	9.7	-8.1	29.2	0	0	0	0			15.5	178.4	0	183.9	15.8				8	2021-01-17T00:00:00	2021-01-17T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	
joshimath	18-01-2021	16.4	3.6	8.8	16.4	2	8.3	-6	36.9	0	0	0	0			12.6	328.4	6.7	172.3	15				7	2021-01-18T00:00:00	2021-01-18T23:45	0.01	Clear	Clear conditions throughout the day.	clear-day	remote	

Fig. 7 Data set screenshots

Fig. 8 Geographical location of the study area, Joshimath, Uttarakhand



ensure the accuracy of predictions. Figure 9 displays the iconography prevalent in Joshimath since 2010. The frequency of rainy days was higher than clear, cloudy, and snowy days combined.

Outliers

Outliers, as described by authors in [74], are data points that significantly deviate from most data in machine learning. They can arise due to measurement fluctuations, new data, or experimental errors, and their presence can distort predictions. This study generated a scatterplot between precipitation and atmospheric features, revealing outliers in the dew variable, as illustrated in Fig. 10ademonstrates this process,

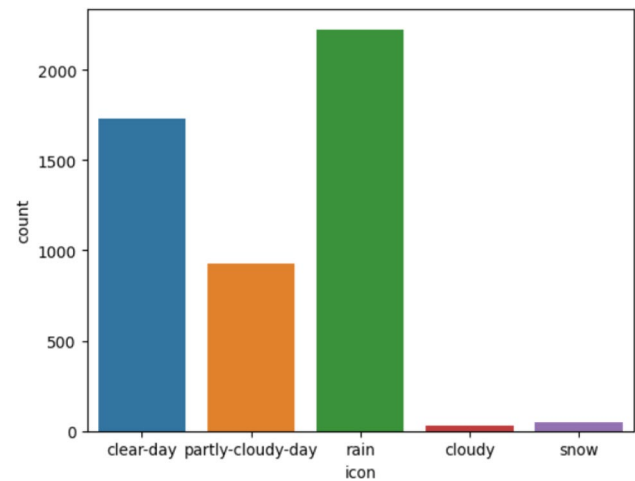
wherein values exceeding the 99th percentile were replaced with the value obtained by multiplying the 99th percentile value by 3. In contrast, values below the 1st percentile were replaced with those obtained by multiplying the 1st percentile value by 0.3, as suggested by the authors in [75].

Correlation Matrix

Correlation is a common metric when evaluating the connection between parameters. It may take on values between -1 and 1, with 0 signifying no interaction and 1 suggesting a strong relationship between the parameters. The associated parameters increase or decrease simultaneously, depending on whether the sign is positive or negative. All

Table 12 Computed values of factors related to rainfall

	Max. Temp	Min. Temp	Dew	Humidity	Precipitation	Wind Dir	Cloud Cover	Solar Energy	Solar Radiation	UV Index	Moon Phase
Mean	20.65	10.09	5.14	58.77	3.47	216.83	34.87	19.58	226.76	7.74	0.48
Standard Deviation	5.74	6.30	9.31	21.68	8.78	77.61	30.55	5.98	69.13	1.83	0.28
Min	-1.20	-9.20	-30.00	7.70	0.00	0.20	0.00	1.00	11.90	0.00	0.00
25%	17.00	4.80	-1.90	40.60	0.00	189.10	6.00	15.40	177.60	7.00	0.25
50%	21.30	10.60	4.50	57.00	0.00	238.60	27.90	19.20	222.00	8.00	0.50
75%	24.40	16.00	13.90	79.70	3.10	266.40	59.70	24.20	279.80	9.00	0.75
Max	34.7	22.1	20.2	97.5	151	359.8	100	32.5	376.2	10	0.98

**Fig. 9** Iconography distribution for 14 years of Joshimath

relevant rainfall variables' correlation matrices are shown in Table 13.

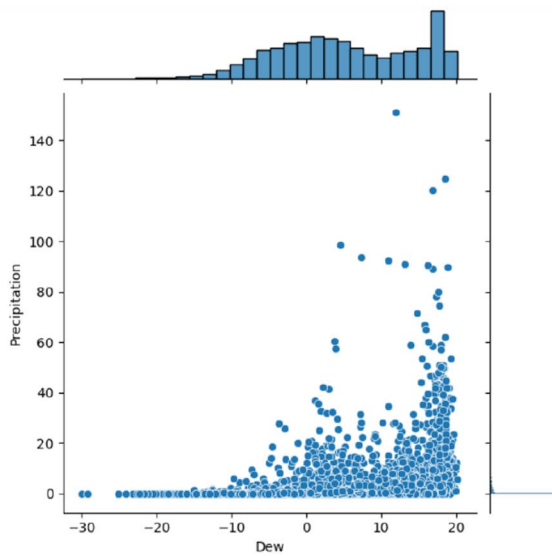
Figure 11 shows how many environmental features are related to precipitation intensity. Here, precipitation cover contributes the most to causing more precipitation, followed by humidity, cloud cover, etc. The UV index is inversed by rainfall amount, followed by solar energy, solar radiation, moon phase, etc.

Experiments

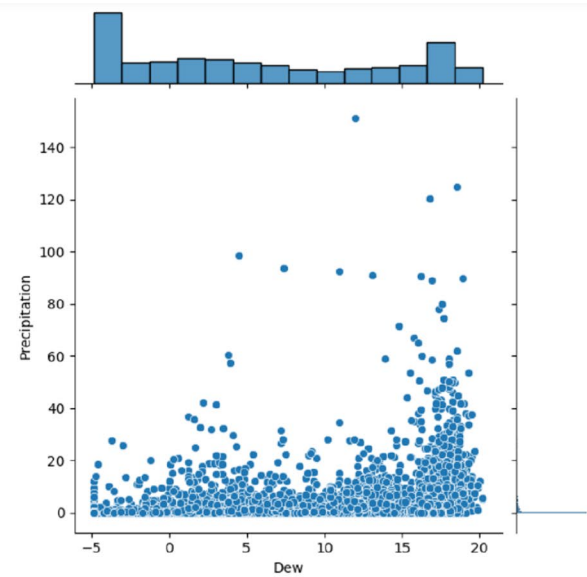
Performance Analysis

The experiments were performed using an NVIDIA Tesla T4 GPU in the freely accessible Google Colab platform. In addition to that, the source code is publicly available through this link: <https://github.com/anuragitpcs/rainfall-prediction-of-joshimath>. Furthermore, as the rainfall forecasting task involves time-series data, a chronological approach is taken to splitting the data, where 80% of the data is used for training and the remaining 20% for testing to avoid temporal leakage. This helps ensure that no future data is incorporated into the training of the model. By applying the time-series split, we ensure that any temporal relationship between the samples remains intact. To make sure that our methodology is as rigorous as possible, we conduct all our experiments by fixing random seeds in all our models, which include not only transformer models but also AutoML approaches. Additionally, our normalization technique is consistently applied across all runs.

Moreover, each model under consideration is subjected to the same train-test split process, and the entire implementation is openly shared to enhance openness. The robustness of the proposed system is gauged through



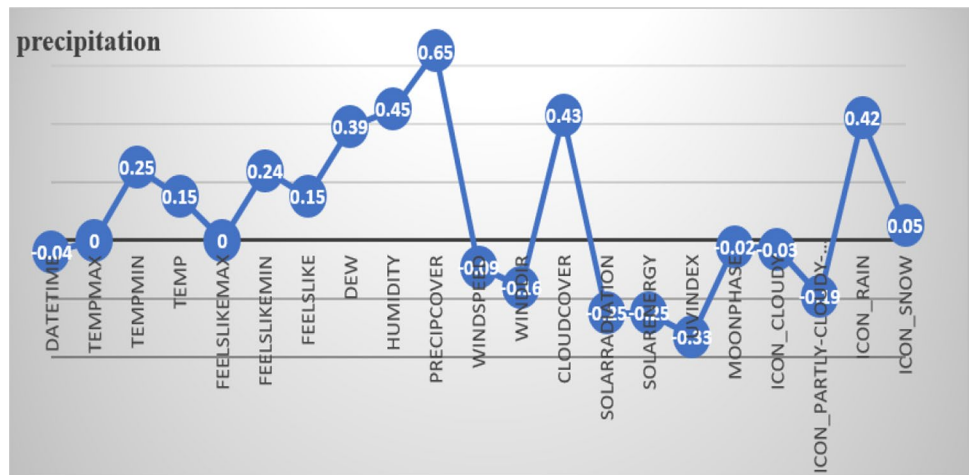
(a) Showing outliers in Dew.



(b) capping and flooring for removing outliers in Dew.

Fig. 10 Scatter plot of outliers**Table 13** Correlation matrix for rainfall affecting variables

	datetime	tempmax	tempmin	temp	feelslikemax	feelslikemin	feelslike	dew	humidity	precip
datetime	1.00	-0.21	-0.17	-0.19	-0.20	-0.19	-0.20	-0.09	-0.04	-0.04
tempmax	-0.21	1.00	0.85	0.95	0.99	0.85	0.95	0.50	-0.02	0.00
tempmin	-0.17	0.85	1.00	0.97	0.86	1.00	0.97	0.84	0.44	0.25
temp	-0.19	0.95	0.97	1.00	0.95	0.96	1.00	0.72	0.25	0.15
feelslikemax	-0.20	0.99	0.86	0.95	1.00	0.86	0.95	0.53	0.01	0.00
feelslikemin	-0.19	0.85	1.00	0.96	0.86	1.00	0.97	0.83	0.44	0.24
feelslike	-0.20	0.95	0.97	1.00	0.95	0.97	1.00	0.73	0.27	0.15
dew	-0.09	0.50	0.84	0.72	0.53	0.83	0.73	1.00	0.82	0.39
humidity	-0.04	-0.02	0.44	0.25	0.01	0.44	0.27	0.82	1.00	0.45
precip	-0.04	0.00	0.25	0.15	0.00	0.24	0.15	0.39	0.45	1.00
precipcover	0.03	0.07	0.44	0.29	0.08	0.41	0.29	0.64	0.70	0.65
windspeed	-0.14	0.50	0.22	0.36	0.46	0.22	0.33	-0.09	-0.38	-0.09
winddir	-0.29	0.35	0.22	0.27	0.36	0.26	0.30	0.07	-0.06	-0.16
cloudcover	0.07	-0.08	0.28	0.13	-0.07	0.26	0.14	0.45	0.56	0.43
solarradiation	-0.01	0.67	0.35	0.53	0.65	0.34	0.51	0.06	-0.33	-0.25
solarenergy	-0.01	0.67	0.35	0.53	0.65	0.34	0.51	0.06	-0.33	-0.25
uvindex	0.04	0.52	0.17	0.35	0.51	0.17	0.33	-0.08	-0.40	-0.33
moonphase	-0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.00	-0.01	-0.02
icon_cloudy	0.03	-0.03	-0.02	-0.03	-0.02	-0.02	-0.03	-0.05	-0.05	-0.03
icon_partly-cloudy-day	-0.06	-0.06	-0.14	-0.11	-0.05	-0.13	-0.11	-0.20	-0.20	-0.19
icon_rain	0.10	0.11	0.41	0.30	0.12	0.38	0.30	0.57	0.58	0.42
icon_snow	0.15	-0.25	-0.20	-0.22	-0.27	-0.21	-0.23	-0.11	0.02	0.05

Fig. 11 Correlation Matrix of atmospheric features with precipitation**Table 14** Performance measurements

Models	MAE	RMSE	R^2
H2O	1.98	5.86	0.58
AutoKeras	7.69	11.19	0.542
TPOT	3.81	8.38	0.37
Autoformer	4.00	9.53	0.17
Informer	7.70	13.71	-0.67
PatchTST	2.27	6.59	0.542
Random Forest	2.27	6.59	0.542
XGBoost	2.28	6.74	0.520
Decision Tree	2.36	6.65	0.533
Bagging	2.31	6.68	0.529
Ada Boost	2.49	7.39	0.525
Ridge Regression	2.89	7.28	0.4411
MLR	2.89	7.28	0.4410
Lasso Regression	2.89	7.28	0.429
KNN	2.51	7.41	0.423

several runs, with the results reported based on the average performance metrics like RMSE, MAE, and R^2 [9, 49, 50], in order to reduce the stochastic variations in model results due to model initialization and training process. In contrast to traditional approaches, where random sampling techniques are employed, our sequential sampling approach guarantees a realistic evaluation of model performance.

Using MAE, RMSE, and R^2 , the evaluation of the AutoML models (H2O, Autokeras, and TPOT), time series transformers (Autoformer, Informer, and PatchTST), and traditional ML models: Random Forest, XGBoost, Decision Tree, Bagging, Ada Boost, Ridge Regression, MLR, Lasso Regression, and KNN regression models depicted and compared in Table 14. AutoML models Autokeras, TPOT, Time-series Transformer models, and traditional ML models were all beaten by H2O models with $R^2 =$

0.58, MAE = 1.98, and RMSE = 5.86 in predicting rainfall based on relevant, chosen environmental data.

As shown in Table 14, the reported R^2 value of the best-performing H2O model is only 0.58. The basic assumptions about R^2 are expected to be close to 0.8–0.9; however, when talking about rainfall forecasts where daily and hourly data are involved, all such generalizations cease to work, and some of the models in ideal conditions generate R^2 values from 0.3–0.6 only. This is due to the fact that the currently used dataset consists of spikes, and the majority of the data cannot be ignored as outliers. The missing spikes generate large squared errors and thus significantly lower the value of R^2 . Hence, the impact of a single missed event will outweigh many accurately captured events. Also, since the vast majority of machine learning algorithms use MSE or RMSE as their loss functions, their optimization process is geared towards accurately predicting the commonly occurring data rather than the outliers.

Rainfall datasets are a perfect example of such cases since: 70–90% observations are low rainfall, and Heavy rainfall occurrences are rare and cannot be ignored. The location of Joshimath is in the Himalayan region, where rainfall patterns are subject to Orographic uplift, local convection, variations in the monsoon, and western disturbances. These factors have the ability to produce rainfall patterns that can seem to occur randomly from a purely time series point of view. Numerical prediction models also find it difficult to predict the weather in such terrain.

Statistical Testing

Some of the tests that we have used when conducting tests for statistical significance between the models are the Paired t-test (p-value) and the Wilcoxon Signed-Rank Test (p-value). The reason behind choosing to use both the paired t-test and the Wilcoxon signed-rank test is based on the fact that both tests are used in comparing related or paired

Table 15 Statistical Comparison of H2O AutoML with Competing Models

Comparison	RMSE Difference	Expected Paired	Approx.	Approx.	Likely
		t-statistic	t-test p-value	Wilcoxon p-value	Conclusion
H2O vs AutoKeras	5.33	> 8.0	< 0.0001	< 0.0001	Highly Significant
H2O vs Autoformer	3.67	> 6.0	< 0.0001	< 0.0001	Highly Significant
H2O vs PatchTST	0.73	1.8–2.5	0.02–0.08	0.03–0.10	Marginal/Moderate
H2O vs Random Forest	0.73	1.8–2.5	0.02–0.08	0.03–0.10	Marginal/Moderate
H2O vs XGBoost	0.88	2.2–3.0	0.005–0.04	0.01–0.05	Significant
H2O vs Bagging	0.82	2.0–2.8	0.01–0.05	0.01–0.06	Significant

models. While the paired t-test is chosen to obtain the ideal data distribution scenario, i.e., data under normal distribution, the Wilcoxon test is selected since the data is highly skewed with some spike points. Table 15 provides the comparison between the proposed model H2O AutoML and six other models, including machine learning and deep learning models, using Paired T-test and Wilcoxon signed rank test statistics, respectively. This comparison is made considering the difference in the RMSE value among the models. From the results presented in Table X, it can be observed that H2O outperforms the other two models, AutoKeras and Autoformer, by a large margin with respect to the RMSE difference, which is 5.33 and 3.67, respectively. These two comparisons produce extremely low p-values (< 0.0001) in both the paired t-test and the Wilcoxon tests. The H2O model also produces a significant improvement compared with XGBoost and Bagging. On the other hand, the variations between H2O and PatchTST, as well as H2O and Random Forest, are quite low (difference RMSE = 0.73). Therefore, the respective p-values lie close to the level of statistical significance, implying only moderate evidence regarding their dominance. Thus, it can be concluded that the H2O AutoML shows the best predictive power compared to other tested algorithms.

Comparative Analysis with Existing Works

The comparative analysis has been extended further to cover current state-of-the-art research on rainfall forecasting models used in various climatic zones, such as India and the Himalayan region. The performance of different machine learning and deep learning models has been compared on the basis of metrics such as RMSE, MAE, and R^2 scores as shown in Table 16. Based on the above comparisons, it can be seen that traditional machine learning techniques like RF and MLR, and deep learning models like LSTM and Transformer models show variations in their performances based on the region and nature of data being considered. The studies done in regions like the Himalayas and India show relatively high errors, signifying the complicated nature of rainfall in these geographical regions. However, the H2O AutoML model designed for forecasting rainfall shows high performance with a much lower RMSE score of 5.86, a comparatively low MAE score of 1.98, and an acceptable R^2 score of 0.58, all for the Joshimath region of the Himalayas. This improvement highlights the effectiveness of automated model selection and ensemble learning in handling complex, region-specific rainfall patterns.

Table 16 Comparative analysis of rainfall prediction models across regions

Study	RMSE	MAE	R^2	Location
Usman et al. (2023)	13.05–13.47	6.62–7.84	–	Semarang
Liyew & Melese (2021)	7.85–8.82	3.58–4.97	–	Ethiopia
Gu et al. (2022)	60.9–68.7	42.1–46.3	0.40–0.53	China
Abiyev et al. (2025)	162.93	51.83	0.831	Nigeria
Nayak et al. (2024)	30.24	–	–	India
Wani et al. (2024)	32.23	21.58	–	North-West Himalayas
Proposed Work	5.86	1.98	0.58	Joshimath

Discussion

The methodological gaps addressed in the current study, rather than simply comparing various models. Firstly, compared to previous rainfall prediction research, which heavily relies on human intervention through manual selection of models and tuning of hyperparameters, this research employs a fully automated AutoML approach (H2O). This makes the study free of any human-caused biases. Secondly, we suggest a comparative methodology framework that would compare various machine learning techniques, including classical models, ensemble methods, and modern transformer-based time series models like Informer, Autoformer, and PatchTST, under the same conditions of data and evaluation metrics. Thirdly, we introduce a process of correlation-based feature selection before implementing any models, thereby combining statistical interpretability with automation, as previous studies have disregarded. Moreover, in addition to other methodological contributions, the study examines the balance between model complexity and automation, demonstrating the effectiveness of AutoML models against advanced transformer-based models when data availability is limited. Lastly, the proposed framework is specifically designed for a high-risk, data-scarce region (Joshimath), making it practically relevant for real-world deployment.

The parameters used in this research originate from the visual crossing, which compiles data from local weather stations and far-away sources like satellites and radar. These factors are then evaluated for their potential influence on the effects of rainfall, as seen in Table 11 for the daily rainfall forecast. Using a dummy variable construction approach, this study has taken all necessary factors and turned non-numerical variables like icons (snow, rain, overcast, and partly cloudy) into numeric ones. This way, all characteristics may be employed. There is no indication of a link between the factors used by authors in [19, 76]. Liyew and Melese [77] used only three ML algorithms, two performance measures (RMSE and MAE) for assessment, and Pearson correlation to demonstrate relationships between variables; nevertheless, they neglected to employ a grid search to enhance the model's accuracy.

In summary, to forecast rainfall, this study analyzed all relevant atmospheric parameters using advanced models, including AutoML models (H2O, AutoKeras, and TPOT), time-series transformers (Autoformer, Informer, and PatchTST), and traditional ML models. Among these, the AutoML model H2O demonstrated the best performance, as assessed by three key indicators: RMSE, MAE, and R^2 .

Conclusion

Rainfall prediction is crucial for agriculture and disaster preparedness, helping mitigate risks from floods and rain-related illnesses, and reducing mortality rates. This study leverages data from the Visual Crossing platform to evaluate advanced models, such as time series transformers (Autoformer, Informer, PatchTST), AutoML frameworks (H2O, AutoKeras, TPOT), and traditional ML models (Random Forest, XGBoost, Decision Tree, Bagging, AdaBoost, Ridge Regression, MLR, Lasso Regression, KNN). Using RMSE, MAE, and R^2 as performance metrics, results show that H2O AutoML outperformed other models, eliminating the need for manual model selection and hyperparameter tuning. Grid search identified Random Forest as the second-best model for rainfall forecasting, confirming its reliability in traditional ML settings.

Funding The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Data Availability The data supporting this study's findings are included in the manuscript.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

Ethics Approval Not applicable.

References

1. T.R. Martha, P. Roy, K.B. Govindharaj, K.V. Kumar, P. Diwakar, V. Dadhwal, Landslides triggered by the June 2013 extreme rainfall event in parts of uttarakhand state, india. *Landslides* **12**, 135–146 (2015)
2. V. Gupta, I. Jamir, V. Kumar, M. Devi, Geomechanical characterisation of slopes for assessing rockfall hazards in the upper yamuna valley, northwest higher himalaya, india. *Himalayan Geol.* **38**(2), 156–170 (2017)
3. S. Parkash, Historical records of socio-economically significant landslides in india. *J. South Asia Disaster Stud.* **4**(2), 177–204 (2011)
4. A.D. Ziegler, R.J. Wasson, A. Bhardwaj, Y. Sundriyal, S. Sati, N. Juyal, V. Nautiyal, P. Srivastava, J. Gillen, U. Saklani, Pilgrims, progress, and the political economy of disaster preparedness-the example of the 2013 uttarakhand flood and kedarnath disaster. *Hydrol. Process.* **28**(24), 5985–5990 (2014)
5. Y. Sundriyal, V. Kumar, N. Chauhan, S. Kaushik, R. Ranjan, M. Punia, Brief communication: The northwest himalaya towns slipping towards potential disaster. *Nat. Hazard.* **23**(4), 1425–1431 (2023)
6. Agarwal, S., Kumar, V., Kumar, S., Sundriyal, Y., Bagri, D.S., Chauhan, N., Kaushik, S., Khan, F., Rana, N.: Identifying

- potential hotspots of land use/land cover change in the last 3 decades, uttarakhand, nw himalaya (2022)
7. S. Zainudin, D.S. Jasim, A.A. Bakar, Comparative analysis of data mining techniques for Malaysian rainfall prediction. *Int. J. Adv. Sci. Eng. Inf. Technol.* **6**(6), 1148–1153 (2016)
 8. P.S. Dutta, H. Tahbiller, Prediction of rainfall using data mining technique over Assam. *Indian J. Comput. Sci. Eng. (IJCSE)* **5**(2), 85–90 (2014)
 9. A. Kusiak, X. Wei, A.P. Verma, E. Roz, Modeling and prediction of rainfall using radar reflectivity data: A data-mining approach. *IEEE Trans. Geosci. Remote Sens.* **51**(4), 2337–2342 (2012)
 10. K.E. Trenberth, Changes in precipitation with climate change. *Climate Res.* **47**(1–2), 123–138 (2011)
 11. V. Tharun, R. Prakash, S.R. Devi, Prediction of rainfall using data mining techniques, in *Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. (IEEE, 2018), pp. 1507–1512
 12. K. Chowdari, R. Girisha, K. Gouda, A study of rainfall over India using data mining, in *2015 International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*. (IEEE, 2015), pp. 44–47
 13. L. Zhao, Prediction model of ecological environmental water demand based on big data analysis. *Environ. Technol. Innovation* **21**, 101196 (2021)
 14. X. Zhou, X. Ruan, H. Wang, G. Zhou, Exploring spatial patterns and environmental risk factors for global maritime accidents: A 20-year analysis. *Ocean Eng.* **286**, 115628 (2023)
 15. Reddy, P.C., Babu, A.S.: Survey on weather prediction using big data analytics. In: 2017 Second International Conference on Electrical, Computer and Communication Technologies (ICECCT), pp. 1–6 (2017). IEEE
 16. S. Kundu, S.K. Biswas, D. Tripathi, R. Karmakar, S. Majumdar, S. Mandal, A review on rainfall forecasting using ensemble learning techniques. *e-Prime-Advances in Electrical Engineering* (Electronics and Energy, 2023), p. 100296
 17. S.D. Latif, N.A.B. Hazrin, C.H. Koo, J.L. Ng, B. Chaplot, Y.F. Huang, A. El-Shafie, A.N. Ahmed, Assessing rainfall prediction models: Exploring the advantages of machine learning and remote sensing approaches. *Alex. Eng. J.* **82**, 16–25 (2023)
 18. R. Vijayan, V. Mareeswari, P. Mohankumar, G.G.K. Srikar, Estimating rainfall prediction using machine learning techniques on a dataset. *Int. J. Sci. Technol. Res. (IJSTR)* **1**(06), 440–445 (2020)
 19. N. Gnanasankaran, E. Ramaraj, A multiple linear regression model to predict rainfall using Indian meteorological data. *Int. J. Adv. Sci. Technol.* **29**(8), 746–758 (2020)
 20. Aswin, S., Geetha, P., Vinayakumar, R.: Deep learning models for the prediction of rainfall. In: 2018 International Conference on Communication and Signal Processing (ICCSP), pp. 0657–0661 (2018). IEEE
 21. A. Garg, H. Pandey, Rainfall prediction using machine learning. *Int. J. Innovative Sci. Res. Technol.* **4**(5), 56–58 (2019)
 22. M. Zounemat-Kermani, O. Batelaan, M. Fadaee, R. Hinkelmann, Ensemble machine learning paradigms in hydrology: A review. *J. Hydrol.* **598**, 126266 (2021)
 23. B. Choubin, E. Moradi, M. Golshan, J. Adamowski, F. Sajedi-Hosseini, A. Mosavi, An ensemble prediction of flood susceptibility using multivariate discriminant analysis, classification and regression trees, and support vector machines. *Sci. Total Environ.* **651**, 2087–2096 (2019)
 24. M. Chaudhari, D. Choudhari, Study of various rainfall estimation & prediction techniques using data mining. *Am. J. Eng. Res.* **6**(7), 137–9 (2017)
 25. I.K. Nti, A. Zaman, O. Nyarko-Boateng, A.F. Adekoya, F. Key-eremeh, A predictive analytics model for crop suitability and productivity with tree-based ensemble learning. *Decision Anal. J.* **8**, 100311 (2023)
 26. C. Thirumalai, K.S. Harsha, M.L. Deepak, K.C. Krishna, Heuristic prediction of rainfall using machine learning techniques, in *2017 International Conference on Trends in Electronics and Informatics (ICEI)*. (IEEE, 2017), pp. 1114–1117
 27. E. Orozco López, D. Kaplan, A. Linhoss, Interpretable transformer neural network prediction of diverse environmental time series using weather forecasts. *Water Resour. Res.* **60**(10), (2024)
 28. R. Singh, D. Lu, K. Tayal, Temporal sequence transformer to advance long-term streamflow prediction, in *NeurIPS Workshop on Tackling Climate Change with ML*. (2024)
 29. M.E.A. Jiang, Transformer-enhanced spatiotemporal neural network for precipitation forecast post-processing. *J. Hydrol.* **630**, 130720 (2024)
 30. H. Liu, J.C.H. Fung, Enhancing quantitative precipitation estimation using transformer-based deep learning. *Earth and Space Sci.* (2024)
 31. M.J. Piran, X. Wang, H.J. Kim, H.H. Kwon, Precipitation nowcasting using transformer-based generative models and transfer learning. *Int. J. Appl. Earth Obs. Geoinf.* **132**, 103962 (2024)
 32. A. Kumar, A. Choubey, S. Mishra, M.H. Kolekar, R. Misra, G.S. Kumar, Automatic machine learning-driven anomaly detection for predictive maintenance and fault diagnosis in urban metro systems. In: *NIELIT's International Conference on Communication, Electronics and Digital Technologies*, pp. 311–326 Springer (2025)
 33. S. Mishra, A. Choubey, B.A. Reddy, R. Misra, Enhancing ev lithium-ion battery management: automated machine learning for early remaining useful life prediction with innovative multi-health indicators: S. mishra et al. *The Journal of Supercomputing* **80**(14), 20813–20860 (2024)
 34. S. Das, A. Dey, A. Pal, N. Roy, Applications of artificial intelligence in machine learning: review and prospect. *International Journal of Computer Applications* **115**(9), (2015)
 35. Y.R. Shrestha, V. Krishna, G. Krogh, Augmenting organizational decision-making with deep learning algorithms: Principles, promises, and challenges. *J. Bus. Res.* **123**, 588–603 (2021)
 36. J. Waring, C. Lindvall, R. Umeton, Automated machine learning: Review of the state-of-the-art and opportunities for healthcare. *Artif. Intell. Med.* **104**, 101822 (2020)
 37. E. LeDell, S. Poirier, H2o automl: Scalable automatic machine learning. In: *Proceedings of the AutoML Workshop at ICML*, vol. 2020 ICML (2020)
 38. M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, F. Hutter, Efficient and robust automated machine learning, in *Advances in neural information processing systems*, p. 28 (2015)
 39. D. Khider, F. Zhu, Y. Gil, autots: Automated machine learning for time series analysis, in *AGU Fall Meeting Abstracts*, vol. 2019, pp. 43–1637 (2019)
 40. N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, A. Smola, Autogluon-tabular: Robust and accurate automl for structured data [arXiv:2003.06505](https://arxiv.org/abs/2003.06505) arXiv preprint (2020)
 41. R.S. Olson, N. Bartley, R.J. Urbanowicz, J.H. Moore, Evaluation of a tree-based pipeline optimization tool for automating data science, in *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 485–492 (2016)
 42. R.S. Olson, J.H. Moore, Tpot: A tree-based pipeline optimization tool for automating machine learning, in *Workshop on Automatic Machine Learning*, pp. 66–74. (PMLR) (2016)
 43. H. Jin, Q. Song, X. Hu, Auto-keras: An efficient neural architecture search system. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1946–1956 (2019)
 44. L. Zimmer, M. Lindauer, F. Hutter, Auto-pytorch: Multi-fidelity metalearning for efficient and robust autodl. *IEEE Trans. Pattern Anal. Mach. Intell.* **43**(9), 3079–3090 (2021)

45. H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Adv. Neural. Inf. Process. Syst.* **34**, 22419–22430 (2021)
46. X. Zhang, K. Yang, L. Zheng, Transformer fault diagnosis method based on timesnet and informer. In: *Actuators*, vol. 13, p. 74 MDPI (2024)
47. Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A Time Series is Worth 64 Words: Long-term Forecasting with Transformers <https://arxiv.org/abs/2211.14730> (2023)
48. T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen, R. Mitchell, I. Cano, T. Zhou, Xgboost: extreme gradient boosting. R package version 0.4-2 1(4), 1–4 (2015)
49. Z. Nabavi, M. Mirzahi, H. Dehghani, P. Ashtari, A hybrid model for back-break prediction using xgboost machine learning and metaheuristic algorithms in chadormalu iron mine. *J. Mining Environ.* **14**(2), 689–712 (2023)
50. M.M.K. Kazemi, Z. Nabavi, M. Khandelwal, Prediction of blast-induced air overpressure using a hybrid machine learning model and gene expression programming (gep): a case study from an iron ore mine. *AIMS Geosci* **9**(2), 357–381 (2023)
51. A.S.T. Srinivas, R. Somula, K. Govinda, A. Saxena, P.A. Reddy, Estimating rainfall using machine learning strategies based on weather radar data. *Int. J. Commun Syst* **33**(13), (2020)
52. J. Brownlee, XGBoost With Python: Gradient Boosted Trees with XGBoost and Scikit-learn. *Machine Learning Mastery*, (2016)
53. G. Seber, A.J. Lee, Departures from assumptions: diagnosis and remedies. In: *Linear Regression Analysis* vol. 329. John Wiley & Sons, (2012)
54. L. Breiman, Statistics department university of california berkeley, ca 94720, in *RANDOM FORESTS*. (2001)
55. A. Cutler, D.R. Cutler, J.R. Stevens, Random forests. *Ensemble machine learning: Methods and applications*, 157–175 (2012)
56. T.K. Ho, Random decision forests, in *Proceedings of 3rd International Conference on Document Analysis and Recognition*, vol. 1, (IEEE, 1995), pp.278–282
57. D. Priyanka, Kumar, Decision tree classifier: a detailed survey. *Int. J. Inform. Decision Sci.* **12**(3), 246–269 (2020)
58. J.R. Quinlan, Ind. decision trees. *Machine Learn.* **1**, 81–106 (1986)
59. A.K. Ahmad, A. Jafar, K. Aljoumaa, Customer churn prediction in telecom using machine learning in big data platform. *J Big Data* **6**(28), <https://doi.org/10.1186/s40537-019-0191-6> (2019)
60. M. Bowles, *Machine Learning in Python: Essential Techniques for Predictive Analysis*. John Wiley & Sons, (2015)
61. N. Oswal, Predicting rainfall using machine learning techniques [arXiv:1910.13827](https://arxiv.org/abs/1910.13827) arXiv preprint (2019)
62. N.K.A. Appiah-Badu, Y.M. Missah, L.K. Amekudzi, N. Ussiph, T. Frimpong, E. Ahene, Rainfall prediction using machine learning algorithms for the various ecological zones of ghana. *IEEE Access* **10**, 5069–5082 (2021)
63. G. Verma, B. Kumar, C. Kumar, A. Ray, M. Khandelwal, Application of krr, k-nn and gpr algorithms for predicting the soaked cbr of fine-grained plastic soils. *Arab. J. Sci. Eng.* (2023). <https://doi.org/10.1007/s13369-023-07962-y>
64. P.N. Tattar, *Hands-On Ensemble Learning with R: A Beginner's Guide to Combining the Power of Machine Learning Algorithms Using Ensemble Techniques*. Packt Publishing Ltd, (2018)
65. T.G. Dietterich, An experimental comparison of three methods for constructing ensembles of decision trees: Bagging, boosting, and randomization. *Mach. Learn.* **40**, 139–157 (2000)
66. L. Breiman, Bagging predictors. *Mach. Learn.* **24**, 123–140 (1996)
67. R. Tibshirani, Regression shrinkage and selection via the lasso. *J. R. Stat. Soc. Ser. B Stat Methodol.* **58**(1), 267–288 (1996)
68. F. Emmert-Streib, M. Dehmer, High-dimensional lasso-based computational regression models: regularization, shrinkage, and selection. *Mach. Learn. Knowledge Extraction* **1**(1), 359–383 (2019)
69. S. Ren, S. Huang, J. Ye, X. Qian, Safe feature screening for generalized lasso. *IEEE Trans. Pattern Anal. Mach. Intell.* **40**(12), 2992–3006 (2017)
70. X. Guo, X. Hu, O.K. Tam, Artificial intelligence and machine learning in fund performance evaluation-a review, in *FinTech Research and Applications: Challenges and Opportunities*. pp.265–304 (2023)
71. S. Streukens, S. Leroi-Werelds, Multicollinearity: An overview and introduction of ridge pls-sem estimation. In: *Partial Least Squares Path Modeling: Basic Concepts, Methodological Issues and Applications*, pp. 183–207 (2023)
72. D. Singh, D. Goyal, P. Biswakarma, V. Joshi, Recent events of land subsidence in alaknanda valley: a case study of sinking holy town joshimath, uttarakhnad, india. *Environment. Development and Sustainability* , 1–29 (2023)
73. Historical weather data for joshimath. Accessed on 5, Jan 2023. <https://www.visualcrossing.com/weather-history/joshimath> (2023)
74. A. Zimek, E. Schubert, H.P. Kriegel, A survey on unsupervised outlier detection in high-dimensional numerical data. *Statistical Anal. Data Mining: The ASA Data Sci. J.* **5**(5), 363–387 (2012)
75. K. Tiwari, K. Mehta, N. Jain, R. Tiwari, G. Kanda, Selecting the appropriate outlier treatment for common industry applications. In: *NESUG Conference Proceedings on Statistics and Data Analysis*, Baltimore, Maryland, USA, pp. 1–5 (2007)
76. S. Prabakaran, P.N. Kumar, P.S.M. Tarun, Rainfall prediction using modified linear regression. *ARNP J. Eng. Appl. Sci.* **12**(12), 3715–3718 (2017)
77. C.M. Liyew, H.A. Melese, Machine learning techniques to predict daily rainfall amount. *J Big Data* **8**(153), <https://doi.org/10.1186/s40537-021-00545-4> (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.